

IPC Through Sockets, Signals and Semaphores (with a Comparison of All IPC Methods)

Table of Contents

Part A: Sockets

1. [What is a Socket?](#)
2. [Types and Domains of Sockets](#)
3. [Client-Server Model](#)
4. [Socket API Functions](#)
5. [Working Example: TCP Server and Client](#)
6. [Unix Domain Sockets](#)

Part B: Signals

7. [What is a Signal?](#)
8. [Common Signals](#)
9. [Working Example: Handling Signals](#)

Part C: Semaphores and Mutexes

10. [Why Synchronization Tools?](#)
11. [Semaphores](#)
12. [Mutex](#)
13. [Working Example: POSIX Semaphore](#)

Part D: Comparison

14. [Comparison of All IPC Mechanisms](#)
15. [Which One Should I Use?](#)
16. [Summary](#)
17. [References](#)

PART A: SOCKETS

1. What is a Socket?

A **socket** is an **endpoint of a two-way communication link**. It is the most flexible IPC mechanism because it works between processes on the **same machine** and also between processes on **different machines across a network**.

Analogy: A socket is like a telephone. To talk, each side needs a phone (socket). One side dials (connects) and the other picks up (accepts). After that, both can talk and listen.

Every network connection you use, whether browsing websites, sending messages, or streaming video, uses sockets.

A network socket is identified by an **IP address + port number**, for example `127.0.0.1:8080`.

- **IP address:** which machine (`127.0.0.1` means "this machine").

- **Port:** which program on that machine (0 to 65535).
-

2. Types and Domains of Sockets

2.1 Socket Domains (Address Families)

Domain	Constant	Used for
IPv4 network	AF_INET	Communication over IPv4 networks
IPv6 network	AF_INET6	Communication over IPv6 networks
Unix domain (local)	AF_UNIX	Communication between processes on the same machine

2.2 Socket Types

Type	Constant	Protocol	Characteristics
Stream socket	SOCK_STREAM	TCP	Reliable, connection-oriented, ordered, no lost or duplicated data
Datagram socket	SOCK_DGRAM	UDP	Connectionless, faster, but packets may be lost or arrive out of order

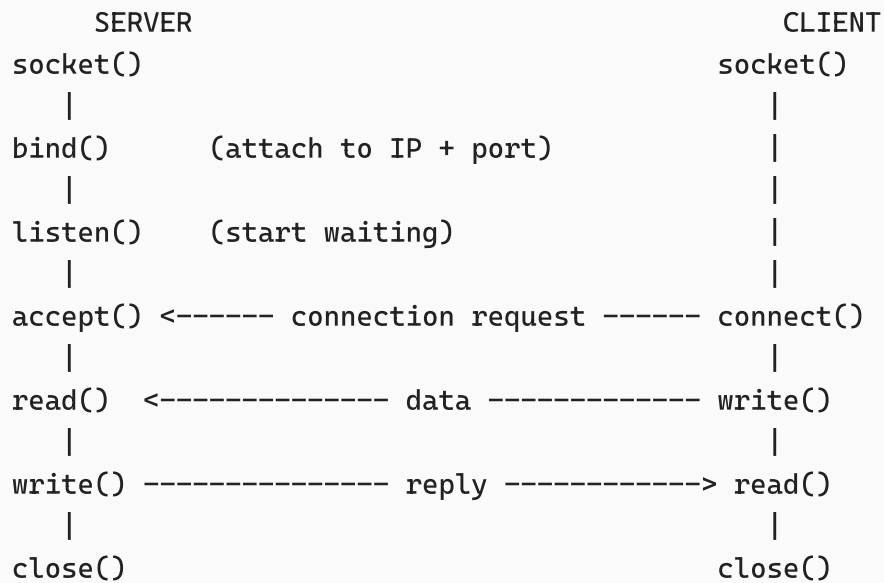
2.3 TCP vs. UDP

Feature	TCP (SOCK_STREAM)	UDP (SOCK_DGRAM)
Connection	Needs connection setup	No connection
Reliability	Guaranteed delivery	Not guaranteed
Order	Preserved	Not preserved
Speed	Slower	Faster
Analogy	Phone call	Postcards
Uses	Web, email, file transfer	Video calls, gaming, DNS

3. Client-Server Model

Sockets are usually used in the **client-server model**:

- **Server**: waits for requests and provides a service.
- **Client**: starts the communication by connecting to the server.



4. Socket API Functions

Header files: `<sys/socket.h>`, `<netinet/in.h>`, `<arpa/inet.h>`, `<unistd.h>`

Function	Used by	Purpose
<code>socket(domain, type, protocol)</code>	Both	Create a socket. Returns a file descriptor.
<code>bind(fd, addr, len)</code>	Server	Attach the socket to an IP address and port
<code>listen(fd, backlog)</code>	Server	Mark the socket as passive, waiting for connections
<code>accept(fd, addr, len)</code>	Server	Accept an incoming connection. Returns a new socket for that client.
<code>connect(fd, addr, len)</code>	Client	Connect to the server
<code>send()</code> / <code>write()</code>	Both	Send data
<code>recv()</code> / <code>read()</code>	Both	Receive data
<code>close(fd)</code>	Both	Close the socket

Byte-order helpers: `htons()` converts a port number to network byte order. `inet_pton()` converts an IP string into binary form.

5. Working Example: TCP Server and Client

The client sends a message, and the server prints it and sends a reply. Both run on the same machine using `127.0.0.1`.

5.1 Server

File: `code/sockets/tcp_server.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define PORT 8080

int main(void) {
    // 1. Create the socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("socket");
        exit(1);
    }

    // Allow quick reuse of the port after restarting
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // 2. Bind to IP + port
    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;    // accept on any local address
    addr.sin_port        = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
        perror("bind");
        exit(1);
    }
}
```

```

}

// 3. Listen (queue up to 3 pending connections)
if (listen(server_fd, 3) == -1) {
    perror("listen");
    exit(1);
}
printf("[Server] Listening on port %d...\n", PORT);

// 4. Accept a client connection (blocks until a client connects)
int client_fd = accept(server_fd, NULL, NULL);
if (client_fd == -1) {
    perror("accept");
    exit(1);
}
printf("[Server] Client connected.\n");

// 5. Receive a message
char buffer[1024] = {0};
ssize_t n = read(client_fd, buffer, sizeof(buffer) - 1);
if (n > 0) {
    printf("[Server] Received: %s\n", buffer);
}

// 6. Send a reply
const char *reply = "Hello client, message received!";
write(client_fd, reply, strlen(reply));
printf("[Server] Reply sent.\n");

// 7. Close
close(client_fd);
close(server_fd);
return 0;
}

```

5.2 Client

File: code/sockets/tcp_client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

```

```

#include <arpa/inet.h>

#define PORT 8080

int main(void) {
    // 1. Create the socket
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == -1) {
        perror("socket");
        exit(1);
    }

    // 2. Describe the server to connect to
    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    // 3. Connect
    if (connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) ==
-1) {
        perror("connect");
        exit(1);
    }
    printf("[Client] Connected to server.\n");

    // 4. Send a message
    const char *msg = "Hello server, this is the client!";
    write(sock, msg, strlen(msg));
    printf("[Client] Sent: %s\n", msg);

    // 5. Receive the reply
    char buffer[1024] = {0};
    ssize_t n = read(sock, buffer, sizeof(buffer) - 1);
    if (n > 0) {
        printf("[Client] Received: %s\n", buffer);
    }

    close(sock);
    return 0;
}

```

5.3 Run

```
gcc tcp_server.c -o tcp_server
gcc tcp_client.c -o tcp_client

./tcp_server      # Terminal 1 (start this first)
./tcp_client      # Terminal 2
```

Sample output:

Terminal 1 (server):

```
[Server] Listening on port 8080...
[Server] Client connected.
[Server] Received: Hello server, this is the client!
[Server] Reply sent.
```

Terminal 2 (client):

```
[Client] Connected to server.
[Client] Sent: Hello server, this is the client!
[Client] Received: Hello client, message received!
```

6. Unix Domain Sockets

When both processes are on the **same machine**, **Unix domain sockets** (`AF_UNIX`) are faster than network sockets because they skip the network stack.

- The address is a **file path** (for example `/tmp/mysocket`) instead of IP and port.
- The same functions (`socket` , `bind` , `listen` , `accept` , `connect`) are used.
- They can also pass **file descriptors** between processes.

Address setup difference:

```
#include <sys/un.h>

struct sockaddr_un addr;
memset(&addr, 0, sizeof(addr));
addr.sun_family = AF_UNIX;
strncpy(addr.sun_path, "/tmp/mysocket", sizeof(addr.sun_path) - 1);
```

Used by: Docker (`/var/run/docker.sock`), X11, PostgreSQL local connections, systemd.

PART B: SIGNALS

7. What is a Signal?

A **signal** is a **short notification** sent to a process to tell it that an event has happened. Signals carry **no data** other than the signal number. They are the simplest form of IPC.

Analogy: A signal is like a doorbell. It tells you someone is there, but it does not tell you what they want.

Signals can be sent by:

- **The kernel** (for example, when a program divides by zero, a `SIGFPE` is sent)
- **The user** (pressing Ctrl+C sends `SIGINT`)
- **Another process** (using the `kill()` function or `kill` command)

When a process receives a signal it can:

1. **Ignore** it (not possible for `SIGKILL` and `SIGSTOP`)
2. **Handle** it with a custom **signal handler** function
3. Let the **default action** happen (usually terminating the process)

8. Common Signals

Signal	Number	Default action	Meaning
<code>SIGINT</code>	2	Terminate	Interrupt from keyboard (Ctrl+C)
<code>SIGQUIT</code>	3	Terminate + core dump	Quit from keyboard (Ctrl+)
<code>SIGKILL</code>	9	Terminate	Force kill. Cannot be caught or ignored.
<code>SIGSEGV</code>	11	Terminate + core dump	Invalid memory access
<code>SIGPIPE</code>	13	Terminate	Write to a pipe with no reader
<code>SIGALRM</code>	14	Terminate	Timer set with <code>alarm()</code> expired
<code>SIGTERM</code>	15	Terminate	Polite request to terminate
<code>SIGCHLD</code>	17 (on x86 Linux)	Ignore	A child process stopped or ended

Signal	Number	Default action	Meaning
SIGSTOP	19 (on x86 Linux)	Stop	Pause the process. Cannot be caught.
SIGUSR1 , SIGUSR2	10, 12 (on x86 Linux)	Terminate	User-defined, free for your own use

Signal numbers can differ between architectures, so it is best to use the **names** in code.

Useful functions and commands:

Function / Command	Purpose
signal(sig, handler)	Register a handler (simple, older)
sigaction()	Register a handler (more reliable, recommended)
kill(pid, sig)	Send a signal to a process
raise(sig)	Send a signal to yourself
pause()	Sleep until a signal arrives
kill -l	List all signals (shell)
kill -SIGUSR1 <pid>	Send a signal from the shell

9. Working Example: Handling Signals

The program installs a handler for SIGUSR1 and SIGINT , then waits. You send it signals from another terminal.

File: code/sockets/signal_demo.c

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Signal handler function
void handler(int sig) {
    if (sig == SIGUSR1) {
        printf("\n[Handler] Received SIGUSR1: hello from another process!\n");
    } else if (sig == SIGINT) {
        printf("\n[Handler] Received SIGINT (Ctrl+C): shutting down cleanly.\n");
    }
}
```

```

        _exit(0);
    }
}

int main(void) {
    struct sigaction sa;
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGUSR1, &sa, NULL);
    sigaction(SIGINT, &sa, NULL);

    printf("Process running. PID = %d\n", getpid());
    printf("From another terminal run: kill -SIGUSR1 %d\n", getpid());
    printf("Press Ctrl+C to exit.\n");

    while (1) {
        pause(); // sleep until a signal arrives
    }
    return 0;
}

```

Compile and run:

```

gcc signal_demo.c -o signal_demo
./signal_demo
# In another terminal:
kill -SIGUSR1 <pid shown above>

```

Sample output:

```

Process running. PID = 4821
From another terminal run: kill -SIGUSR1 4821
Press Ctrl+C to exit.

[Handler] Received SIGUSR1: hello from another process!

[Handler] Received SIGINT (Ctrl+C): shutting down cleanly.

```

Limitations of signals:

- They carry **no data**, only the signal number.

- Ordinary signals are **not queued**: if the same signal arrives twice before the process handles it, it may be delivered only once.
- Inside a handler you should only call **async-signal-safe** functions. `printf` is used here only to keep the demo simple.

PART C: SEMAPHORES AND MUTEXES

10. Why Synchronization Tools?

When processes share resources (for example shared memory), access must be controlled to avoid **race conditions**. Semaphores and mutexes are **coordination tools**. They do not carry data themselves but **control who may proceed and when**.

11. Semaphores

A **semaphore** is an integer variable that can be changed only by two **atomic** (uninterruptible) operations:

Operation	Also called	Meaning
<code>wait(S)</code>	P, down, <code>sem_wait</code>	If $S > 0$, decrement S and continue. If $S == 0$, block until it becomes positive.
<code>signal(S)</code>	V, up, <code>sem_post</code>	Increment S , possibly waking up a waiting process.

```
wait(S):                signal(S):
    while (S <= 0)        S = S + 1;
        ; // wait
    S = S - 1;
```

Types of Semaphores

Type	Value range	Purpose
Binary semaphore	0 or 1	Mutual exclusion (works like a lock)
Counting semaphore	0 to N	Control access to a pool of N identical resources

Using a Semaphore for Mutual Exclusion

```
semaphore mutex = 1;

wait(mutex);
    // CRITICAL SECTION
signal(mutex);
```

Semaphore Analogy

A library has **3 study rooms** (counting semaphore starting at 3). Each student who enters takes a room (`wait` , so the count decreases). When all 3 are taken (count = 0), the next student must wait. When someone leaves (`signal`), the count goes up and a waiting student can enter.

Solving Producer-Consumer with Semaphores

Three semaphores are used:

```
semaphore mutex = 1;    // protects the buffer
semaphore empty = N;    // counts empty slots
semaphore full  = 0;    // counts filled slots

Producer:               Consumer:
wait(empty);            wait(full);
wait(mutex);            wait(mutex);
    add item to buffer    remove item from buffer
signal(mutex);          signal(mutex);
signal(full);           signal(empty);
```

Types of Semaphores in Linux

Kind	API	Notes
POSIX named	<code>sem_open()</code> , <code>sem_wait()</code> , <code>sem_post()</code> , <code>sem_close()</code> , <code>sem_unlink()</code>	Shared between unrelated processes by name
POSIX unnamed	<code>sem_init()</code> , <code>sem_destroy()</code>	For threads, or for processes if placed in shared memory
System V	<code>semget()</code> , <code>semop()</code> , <code>semctl()</code>	Older, uses semaphore <i>sets</i>

12. Mutex

A **mutex** (mutual exclusion lock) is like a binary semaphore with one important difference: **it has an owner**. Only the thread that locked it may unlock it.

Feature	Mutex	Binary Semaphore
Purpose	Locking (mutual exclusion)	Signalling and locking
Ownership	Yes: the locker must unlock	No: any process can signal
Typical use	Protect a critical section	Signal events between processes

Basic usage with threads (`pthread`):

```
pthread_mutex_lock(&lock);  
    // critical section  
pthread_mutex_unlock(&lock);
```

A mutex is mainly used between **threads** of the same process. It can be used between processes only when placed in shared memory with the right attributes, so semaphores are more common for process-level synchronization.

13. Working Example: POSIX Semaphore

A parent and child both print lines. A **named semaphore** makes sure only one of them prints its block at a time, so the lines do not get mixed.

File: `code/sockets/semaphore_demo.c`

```
#include <stdio.h>  
#include <stdlib.h>  
#include <fcntl.h>  
#include <semaphore.h>  
#include <sys/stat.h>  
#include <sys/wait.h>  
#include <unistd.h>  
  
#define SEM_NAME "/ipc_demo_sem"  
  
static void print_block(const char *who) {  
    for (int i = 1; i <= 3; i++) {  
        printf("[%s] line %d\n", who, i);  
    }  
}
```

```

        fflush(stdout);
        usleep(200000);          // 0.2 second, to make interleaving visible
    }
}

int main(void) {
    // Create a binary semaphore with initial value 1
    sem_t *sem = sem_open(SEM_NAME, O_CREAT, 0666, 1);
    if (sem == SEM_FAILED) {
        perror("sem_open");
        exit(1);
    }

    pid_t pid = fork();

    if (pid == 0) {
        // ---- CHILD ----
        sem_wait(sem);              // enter critical section
        print_block("Child ");
        sem_post(sem);              // leave critical section
        sem_close(sem);
        exit(0);
    } else {
        // ---- PARENT ----
        sem_wait(sem);              // enter critical section
        print_block("Parent");
        sem_post(sem);              // leave critical section

        wait(NULL);
        sem_close(sem);
        sem_unlink(SEM_NAME);      // remove the semaphore
    }
    return 0;
}

```

Compile and run:

```

gcc semaphore_demo.c -o semaphore_demo -pthread
./semaphore_demo

```

Sample output (the process that gets the semaphore first prints its whole block):

```

[Parent] line 1
[Parent] line 2
[Parent] line 3

```

```
[Child ] line 1
[Child ] line 2
[Child ] line 3
```

The order may be Child first on some runs, but the lines from the two processes **never mix**. Try removing the `sem_wait` and `sem_post` calls and see how the output interleaves.

PART D: COMPARISON

14. Comparison of All IPC Mechanisms

Mechanism	Speed	Complexity	Data Size	Direction	Related processes only?	Across network?	Builds on sys
Shared Memory	Fastest	High	Large	Two-way	No	No	No (ma
Message Queue	Medium	Medium	Small to medium	Two-way	No	No	Yes
Pipe (anonymous)	Fast	Low	Small to medium	One-way	Yes	No	Yes (blc
FIFO (named pipe)	Fast	Low	Small to medium	One-way	No	No	Yes (blc
Unix Domain Socket	Fast	Medium	Any	Two-way	No	No	Yes
Network Socket	Slower	Medium to high	Any	Two-way	No	Yes	Yes
Signal	Fast	Low	None (only signal number)	One-way	No	No	n/a
Semaphore / Mutex	Fast	Medium	None (control	n/a	No	No	It is syn

Mechanism	Speed	Complexity	Data Size	Direction	Related processes only?	Across network?	Builds on
			only)				

Quick Facts

- **Fastest:** shared memory (no copying)
- **Simplest:** pipes and signals
- **Most flexible:** sockets (local and network)
- **Best for notifications:** signals
- **Best for coordination:** semaphores and mutexes

15. Which One Should I Use?

```

Need to communicate between machines over a network?
|
+-- YES --> Network Sockets (TCP/UDP)
|
+-- NO (same machine)
    |
    | Need to move LARGE amounts of data very fast?
    |     +-- YES --> Shared Memory (+ semaphores for sync)
    |
    | Parent and child process, simple one-way flow?
    |     +-- YES --> Anonymous Pipe
    |
    | Unrelated processes, simple byte stream?
    |     +-- YES --> FIFO (Named Pipe)
    |
    | Discrete, typed messages / priorities?
    |     +-- YES --> Message Queue
    |
    | Two-way local client-server?
    |     +-- YES --> Unix Domain Socket
    |
    | Just need to notify a process that something happened?
    |     +-- YES --> Signal

```


16. Summary

- **Sockets** are the most versatile IPC method. They work locally (Unix domain) and across networks (TCP/UDP) using the client-server model.
 - **Signals** are lightweight notifications with no data. A process can handle, ignore, or accept the default action for most signals.
 - **Semaphores and mutexes** do not transfer data. They **synchronize** processes so that shared resources are accessed safely.
 - No single IPC method is best for everything. The right choice depends on **speed, data size, whether processes are related, and whether a network is involved**.
-

17. References

1. Silberschatz, Galvin, Gagne. *Operating System Concepts* (Chapters 3 and 6).
2. Stevens, W. R. *Unix Network Programming*.
3. Kerrisk, M. *The Linux Programming Interface* (Chapters 20 to 22 on signals, 53 on POSIX semaphores, 56 to 61 on sockets).
4. Linux manual pages: `man 2 socket`, `man 7 signal`, `man 7 sem_overview`, `man 7 unix`.
5. Beej's Guide to Network Programming: <https://beej.us/guide/bgnet/>